

Documentación Técnica

Sistema de Reservas para Salas de Reuniones y Cowork

Versión: 1.0

Fecha: Enero 2026

Realizado por: Virginia Díaz

1. Introducción

1.1 Propósito del Documento

Este documento describe la arquitectura técnica del Sistema de Reservas de Espacios, diseñado para permitir la gestión eficiente de reservas de salas de reuniones y espacios de cowork en un entorno cloud escalable y seguro.

1.2 Alcance

El sistema permite:

- Gestión completa de reservas (crear, consultar, modificar, cancelar)
- Consulta de disponibilidad en tiempo real
- Autenticación y autorización de usuarios
- Procesamiento de pagos
- Sistema de notificaciones automáticas
- Escalabilidad automática según demanda

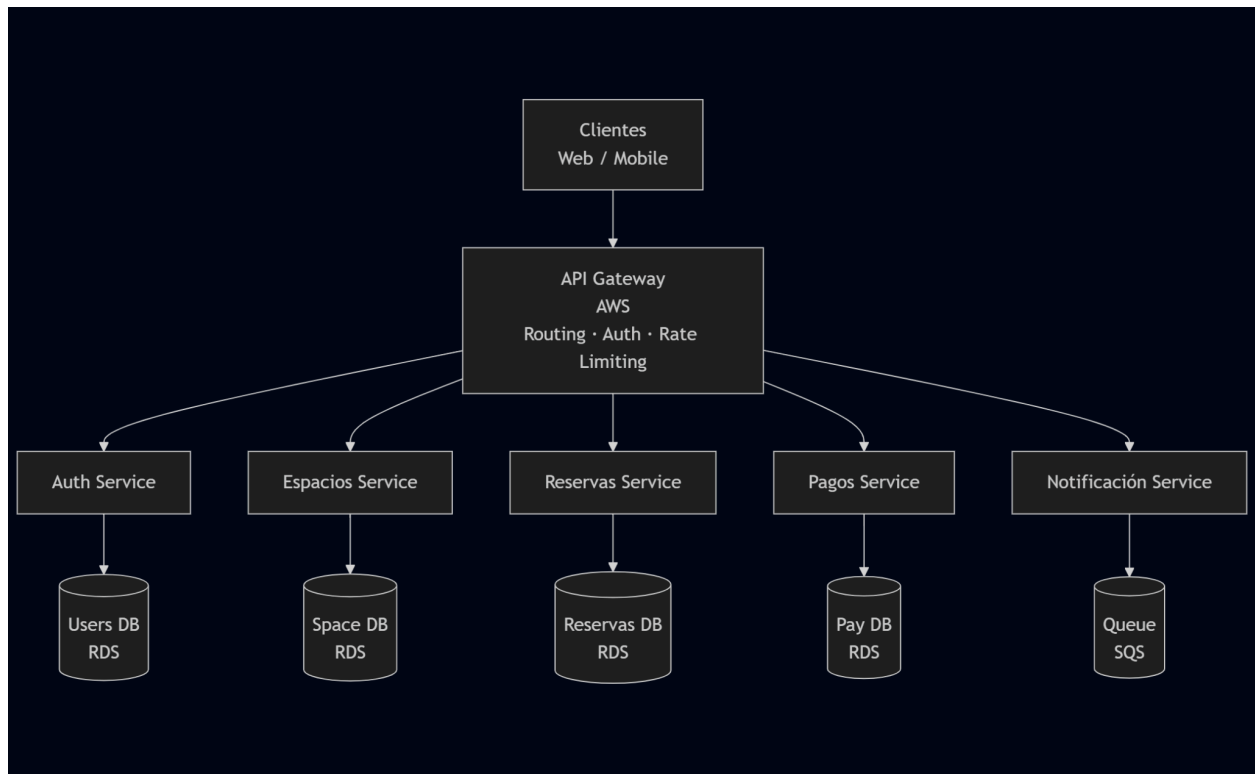
2. Arquitectura General

2.1 Patrón Arquitectónico

El sistema utiliza una **arquitectura de microservicios** que proporciona:

- Desacoplamiento de componentes
- Escalabilidad independiente por servicio
- Facilidad de mantenimiento y despliegue
- Tolerancia a fallos
- Flexibilidad tecnológica

2.2 Componentes de Alto Nivel



3. Servicios Principales

3.1 Servicio de Autenticación

Responsabilidad: Gestión de identidad y acceso

Funcionalidades:

- Registro de usuarios
- Login/Logout
- Generación y validación de tokens JWT
- Renovación de tokens
- Gestión de roles y permisos

Endpoints principales:

POST /api/auth/register
POST /api/auth/login
POST /api/auth/logout
POST /api/auth/refresh
GET /api/auth/validate

Base de datos: PostgreSQL (AWS RDS)

Modelo de datos:

usuarios:

- id (UUID)
- email (string, unique)
- password_hash (string)
- nombre (string)
- tipo (enum: CLIENTE, ADMIN, OPERADOR)
- created_at (timestamp)
- updated_at (timestamp)

3.2 Servicio de Espacios

Responsabilidad: Catálogo y gestión de espacios

Funcionalidades:

- CRUD de espacios
- Consulta de disponibilidad
- Gestión de características
- Búsqueda y filtrado

Endpoints principales:

GET /api/espacios
GET /api/espacios/{id}
GET /api/espacios/{id}/disponibilidad
POST /api/espacios
PUT /api/espacios/{id}
DELETE /api/espacios/{id}

Modelo de datos:

espacios:

- id (UUID)
- nombre (string)
- tipo (enum: SALA_REUNIONES, COWORK, OFICINA_PRIVADA)
- capacidad (integer)
- precio_hora (decimal)
- recursos (jsonb: proyector, pizarra, etc.)
- disponible (boolean)
- created_at (timestamp)

3.3 Servicio de Reservas

Responsabilidad: Gestión del ciclo de vida de reservas

Funcionalidades:

- Creación de reservas
- Validación de disponibilidad
- Modificación de reservas
- Cancelación con políticas
- Historial de reservas

Endpoints principales:

POST /api/reservas

GET /api/reservas/{id}

GET /api/reservas/usuario/{userId}

PUT /api/reservas/{id}

DELETE /api/reservas/{id}

Modelo de datos:

reservas:

- id (UUID)
- usuario_id (UUID, FK)
- espacio_id (UUID, FK)
- fecha_inicio (timestamp)
- fecha_fin (timestamp)
- estado (enum: PENDIENTE, CONFIRMADA, CANCELADA, COMPLETADA)
- precio_total (decimal)
- created_at (timestamp)
- updated_at (timestamp)

3.4 Servicio de Pagos

Responsabilidad: Procesamiento de transacciones

Funcionalidades:

- Integración con Paypal
- Procesamiento de pagos
- Gestión de reembolsos

- Registro de transacciones

Endpoints principales:

POST /api/pagos/procesar

POST /api/pagos/reembolsar

GET /api/pagos/transaccion/{id}

3.5 Servicio de Notificaciones

Responsabilidad: Comunicación con usuarios

Funcionalidades:

- Envío de emails (AWS SES)
- Envío de SMS (AWS SNS)
- Notificaciones push
- Recordatorios automáticos

Comunicación: Event-driven mediante AWS SQS

Tipos de notificaciones:

- Confirmación de reserva
- Recordatorio 24h antes
- Cancelación de reserva
- Cambios en reserva

4. Tecnologías Utilizadas

4.1 Backend

Tecnología	Versión	Propósito
Python	3.11	Lenguaje y Runtime principal
FastAPI	0.110+	Framework web
Pydantic	2.x	Lenguaje tipado
PostgreSQL	15.x	Base de datos relacional
SQLAlchemy	2.x	ORM

Redis	7.x	Caché y sesiones
Pytest	8.x	Testing

4.2 Frontend

Tecnología	Versión	Propósito
React	18.x	Framework UI
TypeScript	5.x	Lenguaje tipado
Tailwind CSS	3.x	Estilos
React Query	4.x	Gestión de estado servidor
Vite	5.x	Build tool

4.3 Infraestructura AWS

Servicio	Propósito
ECS/Fargate	Orquestación de contenedores
ECR	Registro de imágenes Docker
RDS PostgreSQL	Bases de datos
ElastiCache Redis	Caché distribuido
API Gateway	Punto de entrada y routing
Application Load Balancer	Distribución de carga
S3	Almacenamiento de archivos estáticos
CloudFront	CDN
SQS	Cola de mensajes
SNS	Notificaciones
SES	Envío de emails

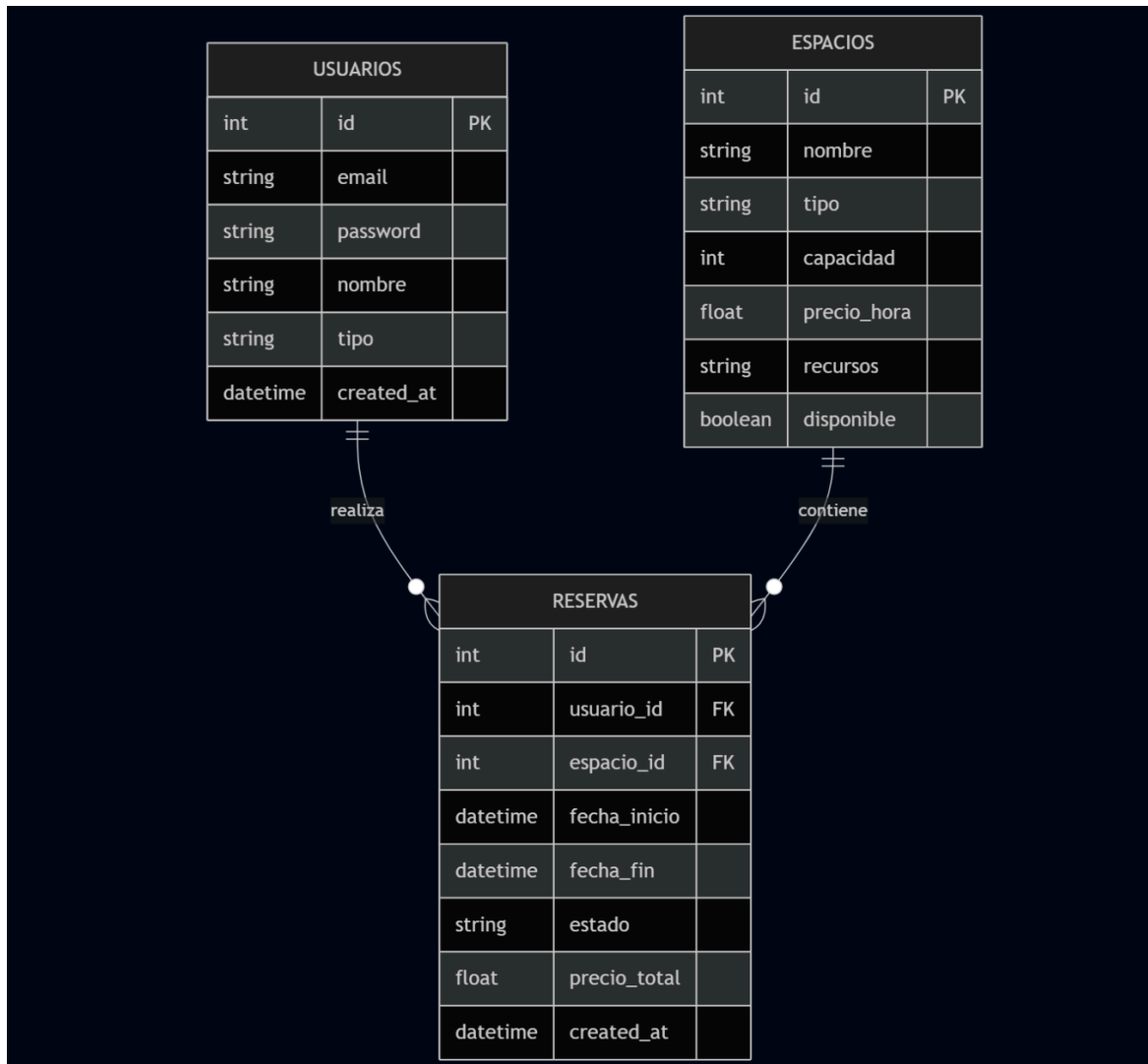
CloudWatch	Monitoreo y logs
Secrets Manager	Gestión de secretos
IAM	Gestión de accesos

4.4 DevOps

Herramienta	Propósito
Docker	Contenedorización
Terraform	Infrastructure as Code
GitHub Actions	CI/CD
SonarQube	Análisis de código
Postman/Newman	Testing de APIs

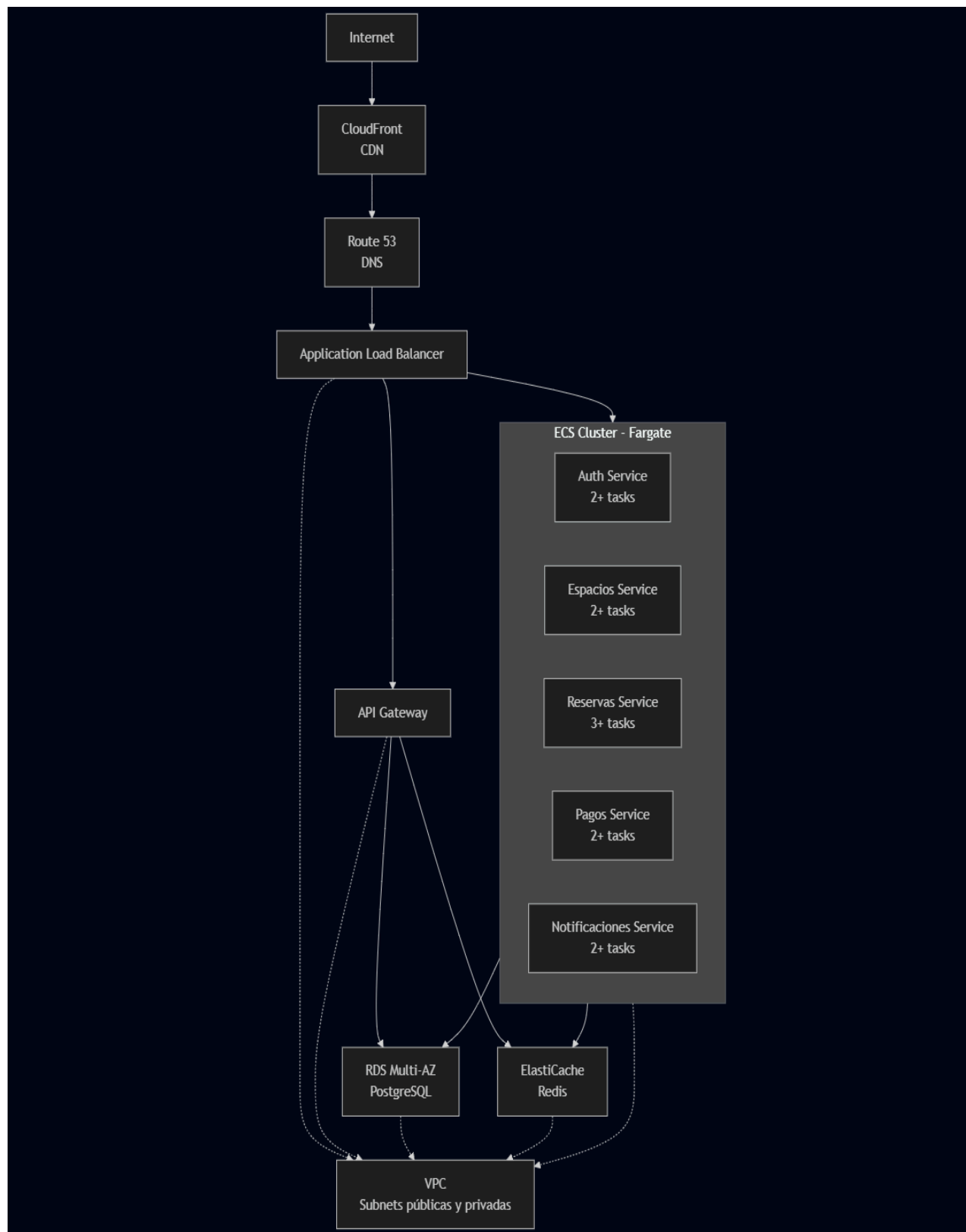
5. Estructura de Datos

5.1 Diagrama Entidad-Relación



6. Despliegue en AWS

6.1 Arquitectura AWS



6.2 Pasos de Despliegue

1. Preparar infraestructura con Terraform
2. Construir y subir imágenes Docker
3. Desplegar servicios en ECS
4. Configurar Auto Scaling

7. Seguridad

7.1 Autenticación y Autorización

JWT Token Structure:

```
{
  "sub": "user-uuid",
  "email": "user@example.com",
  "role": "CLIENTE",
  "iat": 1234567890,
  "exp": 1234571490
}
```

Flujo de autenticación:

1. Usuario envía credenciales
2. Servicio valida en DB
3. Genera JWT firmado
4. Cliente incluye token en header: `Authorization: Bearer <token>`
5. API Gateway valida token antes de enrutar

8. Monitoreo y Observabilidad

8.1 Métricas

Dashboards:

- Overview del sistema
- Health checks por servicio
- Performance metrics
- Business metrics (reservas creadas, canceladas)

8.2 Logging

Estructura de logs:

```
{
  "timestamp": "2026-01-12T10:30:00Z",
  "level": "INFO",
  "service": "reservas-service",
  "traceId": "abc123",
  "userId": "user-uuid",
  "action": "CREATE_RESERVA",
  "message": "Reserva creada exitosamente",
  "metadata": {
    "reservId": "res-uuid",
    "espaciId": "esp-uuid"
  }
}
```

Log aggregation: CloudWatch Logs Insights

8.3 Alertas

Configuración de alertas:

- CPU > 80% por 5 minutos
- Errores 5xx > 1% en 5 minutos
- Latencia P95 > 1000ms
- Health check failures
- Base de datos con conexiones > 90%

Canales de notificación:

- SNS → Email
- SNS → Slack webhook
- PagerDuty para incidentes críticos

9. Apéndices

9.1 Referencias

- [Documentación AWS ECS](#)
- [PostgreSQL Performance](#)
- [Microservices Patterns](#)